Simulation of Process Scheduling Algorithms

Operating Systems 1 – Scheduling

Decalf Kobe De Craene Zenn

March 25, 2026

Abstract

In this report we present our findings achieved by creating and simulating different scheduling algorithms. We implemented FCFS, RR, SPN, SRT, HRRN and MLFB. The metrics for performance are turnaround time (TAT), normalized TAT (nTAT) and the average waiting time. The chosen parameters for MLFB will also be justified.

1 Experimental Setup

1.1 Implemented Scheduling Algorithms

We implemented the following scheduling algorithms:

- First Come, First Serve (FCFS)
- Round Robin (RR) with time slices 2, 4, and 8
- Shortest Process Next (SPN)
- Shortest Remaining Time (SRT)
- Highest Response Ratio Next (HRRN)
- 2 versions of Multi-Level Feedback (MLFB) with 5 levels

1.2 Datasets

We used the provided XML datasets containing processes with:

- process ID,
- arrival time,
- service time.

The experiments were performed on the following datasets:

- Dataset A: 10000 processes
- Dataset B: 20000 processes
- Dataset C: 50000 processes

1.3 Measured Metrics

For each scheduling algorithm and each dataset, we computed:

- average turnaround time (TAT),

- average normalized turnaround time (nTAT),
- average waiting time (T_w).

In addition, for two selected datasets, we created:

- one $nTAT(T_s)$ graph,
- one $T_w(T_s)$ graph,

where each point represents the arithmetic mean over one percentile of service times.

2 Design and Implementation

2.1 Simulator Structure

The fundamental part of our simulator is a *Process*, multiple processes are stored in a *ProcessList*. For reading in the XML input we use a class called *ProcessReader*, which returns a *ProcessList*. We use an abstract class *Algorithm* with the basics that each algorithm would need, every executor uses this via inheritance. Then, for each different algorithm, we have a queue and an executor. The executor follows the same 3 steps for each different algorithm. First, it looks for new arrivals, then it selects a next process if necessary and lastly, it executes the current process. The queue stores all the arrived processes and the logic for selecting the next process is also found here. Time progression in our simulator is very simple, every time the loop gets executed, it represents one jiffy. All statistics are stored in the *Process* itself, which means the processes are stored in a csv file and used for evaluation later. We used `python` (`matplotlib`) to render the graphs. Our graphs have been smoothed out using `statsmodels.nonparametric.smoothers_lowess`.

3 MLFB Configuration

We used an MLFB scheduler with 5 levels. The chosen parameters are:

Parameter	Value / Description
Number of levels	5
Time quantum per level	- 1, 2, 4, 8, 16 - 8, 16, 32, 64, 128
New process level	highest priority queue
Demotion rule	after full quantum usage
Promotion rule	not implemented

Table 1: Chosen MLFB parameters.

3.1 Motivation

The MLFB scheduler uses 5 levels. We used two versions of this algorithm. For the first version we used `(1, 2, 4, 8, 16)` for the time quantums, as a baseline. The optimal configuration with our algorithm was `(8, 16, 32, 64, 128)`. This decreases the normalized turnaround time and average waiting time for shorter processes while not increasing the average waiting time for long processes too much. New processes are added to the highest priority queue and the demotion rule we used is when the process reaches full quantum usage. This is standard. We didn't implement a promotion rule, which could lead to starvation of longer processes if lots of short processes keep arriving.

4 Results

4.1 Global Measurements

Dataset	Algorithm	Avg. TAT	Avg. nTAT	Avg. T_w
10000	FCFS	540.66	26.94	441.01
10000	RR-2	542.72	5.54	443.07
10000	RR-4	542.64	5.80	442.99
10000	RR-8	542.74	6.40	443.09
10000	SPN	301.58	6.27	201.93
10000	SRT	246.54	1.62	146.90
10000	HRRN	375.74	7.71	276.09
10000	MLFB1	542.91	4.64	443.26
10000	MLFB2	546.33	4.30	446.68
20000	FCFS	536.42	22.97	435.64
20000	RR-2	544.03	5.39	443.25
20000	RR-4	543.93	5.60	443.15
20000	RR-8	543.67	6.06	442.89
20000	SPN	297.24	5.70	196.46
20000	SRT	246.98	1.61	146.20
20000	HRRN	373.03	7.12	272.25
20000	MLFB1	545.42	4.61	444.64
20000	MLFB2	545.94	4.25	445.16
50000	FCFS	505.33	23.26	405.80
50000	RR-2	498.41	5.08	398.89
50000	RR-4	498.50	5.30	398.98
50000	RR-8	498.57	5.76	399.05
50000	SPN	289.58	5.85	190.06
50000	SRT	235.85	1.58	136.32
50000	HRRN	353.07	7.26	253.54
50000	MLFB1	497.94	4.29	398.41
50000	MLFB2	497.62	3.95	398.09

Table 2: Global measurements for all algorithms and datasets.

4.2 Graphs for Selected Datasets

4.2.1 Dataset 10000

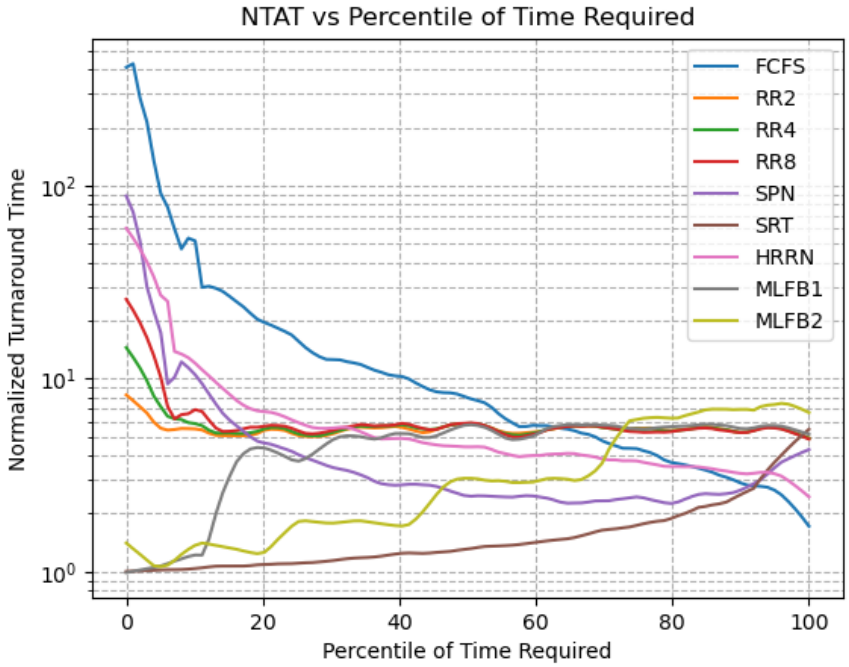

Figure 1: $nTAT(T_s)$ for Dataset 10000.

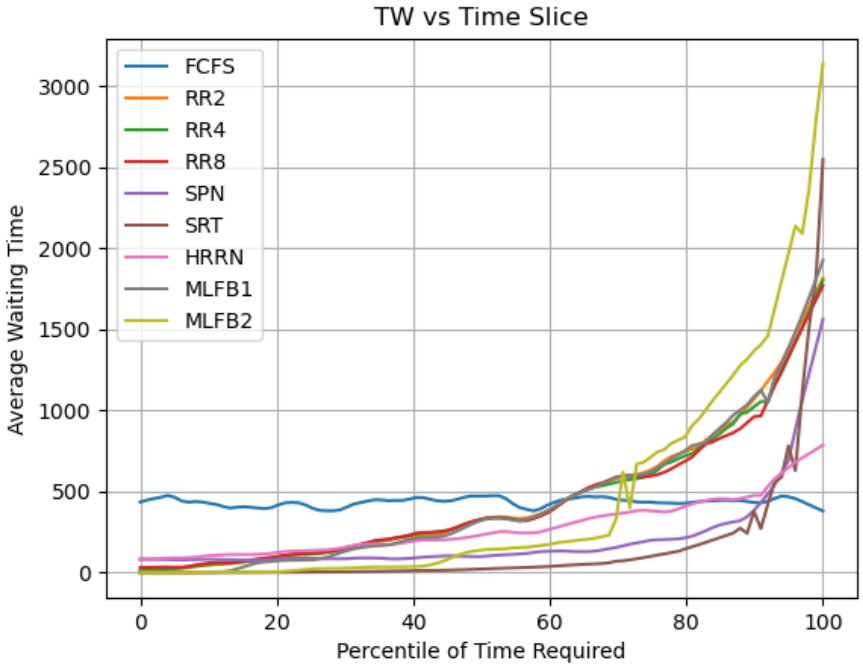

Figure 2: $T_w(T_s)$ for Dataset 10000.

4.2.2 Dataset 50000

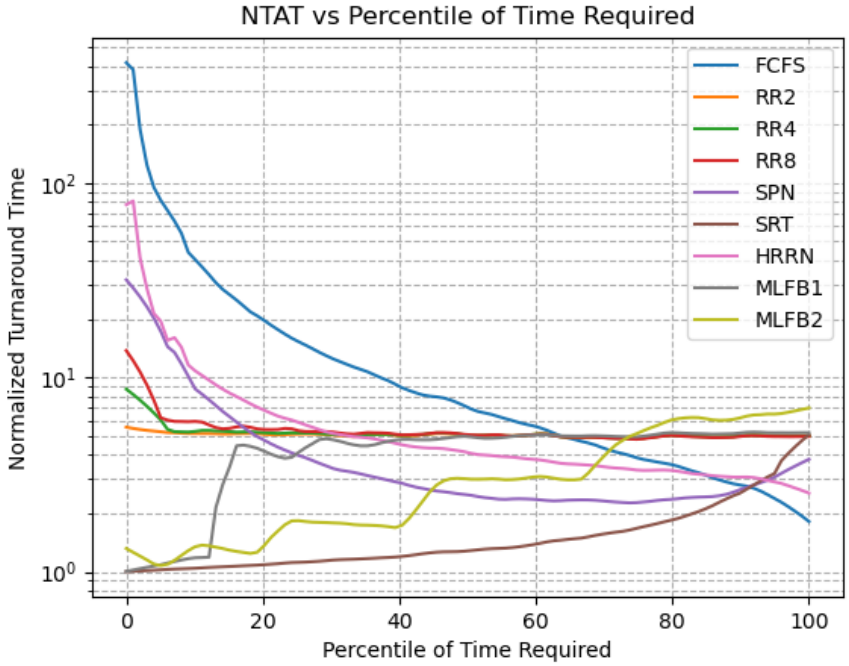

Figure 3: $nTAT(T_s)$ for Dataset 50000.

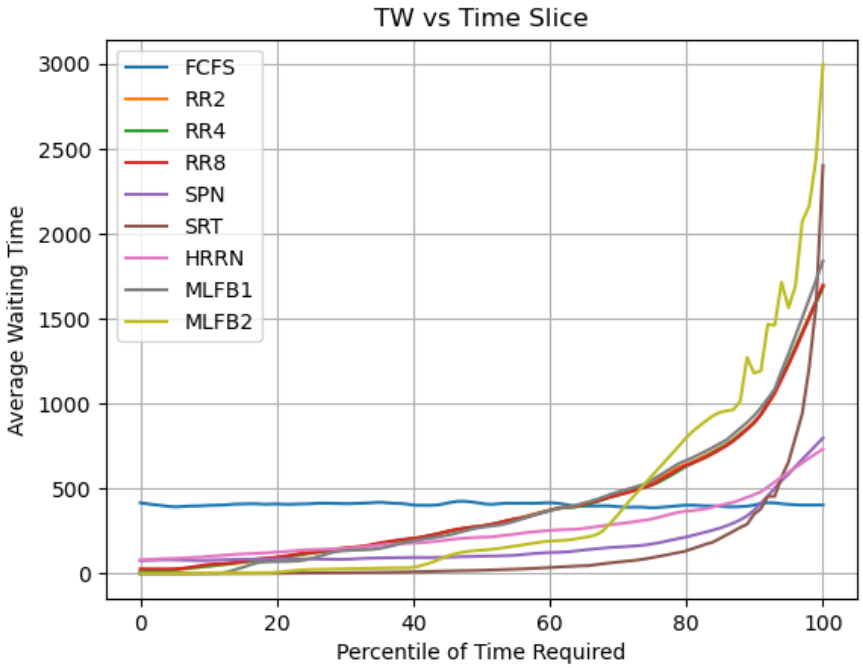

Figure 4: $T_w(T_s)$ for Dataset 50000.

5 Evaluation and Comparison

5.1 Overall Comparison

The algorithm that minimizes both nTAT and T_w is SRT. For short and medium-length processes the nTAT is noticeably shorter than all other algorithms. Only for long processes (that have a percentile of time greater than 90%) is the nTAT and T_w better. From the graph with the normalized turnaround time we derive SRT and MLFB strongly favour short jobs. The average waiting time for all algorithms increase with the percentile of time required, except for FCFS. The waiting time for that algorithm is constant.

5.2 Behaviour Across Service-Time Percentiles

For the shorter jobs, FCFS is the worst choice (by far), the nTAT of FCFS is very large in the lower percentiles because of the fact that very short jobs sometimes need to wait a very long time because they arrived behind longer work. SPN, SRT and RR with smaller quanta keep the nTAT much lower than FCFS and this shows that those algorithms favour short processes over longer ones.

For medium jobs, all the curves seem to converge: RR2, RR4, RR8, HRRN and MLFB cluster together in a small band. This shows that these algorithms treat these jobs more balanced, no extremes are found around these.

For the longest jobs, the info of the shorter jobs reverses. The algorithms that favour shorter jobs give the largest waiting time on long processes (see waiting time plots). This does not mean that algorithms with higher waiting times (like SRT, SPN and MLFB), do not necessarily give the worst nTAT for the long jobs.

Overall, the graphs show that some algorithms clearly favour one kind of job sizes. FCFS favours long jobs, while SRT, SPN and MLFB favours the shorter ones. RR and HRRN are the most balanced of the algorithms, since those lie closer to the middle.

5.3 Round Robin Analysis

As we can see from the 2 different time slices used with Round Robin, choosing the time-slice is something that needs to be thought about. A small quantum will improve the responsiveness so short processes will benefit from this because they are able to finish quicker since it is their turn faster. As the quantum gets bigger, Round Robin begins to behave like FCFS as we can see in both the nTAT and waiting-time curves. The only reason for choosing a larger quantum is that it reduces the overhead and fragmentation but because we simulate it here, overhead is already minimal. To summarize, we can see the trade-off of Round Robin very clearly here. Smaller quanta improves the short-job responsiveness but has an inefficiency for long-running processes while larger quanta has a lower overhead, but weakens the responsiveness advantage RR has over FCFS.

6 Comparison with Theory and Conclusion

When we compare the results of our simulation with the theory we see the graphs generally follow the same trends. The biggest difference is for MLFB the theoretical graph goes to infinity for short processes, while our version stays around zero. This happens because we use smaller time quanta. The theoretical graph of MLFB also shows less 'steps'. These would disappear when using more levels. For the normalized turnaround time all theoretical algorithms except SRT go to infinity for very short processes, this is not as extreme with our findings. The waiting

time in the theoretical algorithms goes to infinity for long processes except for FCFS, this is also less extreme for us.

The best performing algorithm is SRT. It beats all other algorithms in both normalized turnaround time and average waiting time, except for processes that use more than 90 percent of time required. For those processes FCFS, HRRN and SPN perform better. The current implementation of SRT could lead to starvation for those processes. This is a known trade-off in the scheduling world: fairness vs efficiency.